

Module 3 Study Guide: Applied Uninformed Search

Understanding graph theory conceptually is only the first step. You must translate these abstract models into performant engine code. In this guide, we dissect two Python scripts that transition our theoretical Uninformed Search algorithms into functional, memory-aware implementations.

You are examining **Uninformed Search**. These are “blind” algorithms. They possess no heuristic data; they do not know what direction the goal lies in, only what the goal looks like when they stumble upon it.

1. **m03.py**: A pure-logic, terminal-based implementation proving the baseline algorithmic mechanics of BFS and DFS.
2. **m03_gui.py**: A real-time Pygame implementation that visualises the frontier expansion, step counts, and the catastrophic memory scaling of Adjacency Matrices.

Block 1: Pure Logic & The Deque Mandate

Review `m03_terminal.py`. You will notice an explicit reliance on Python's `collections.deque` for Breadth-First Search.

```
from collections import deque

def breadth_first_search(graph: Graph, start: Node, goal: Node) -> list[Node]:
    queue = deque([start])
    visited = {start}

    while queue:
        path = queue.popleft() # O(1) operation
        # ... expansion logic ...
```

The Architectural Reality: If you use a standard Python list and call `path = list.pop(0)`, the CPU must physically shift every remaining pointer in contiguous memory down by one index. That is an $O(N)$ operation. Inside a game loop processing 10,000 nodes, your algorithm degrades to $O(N^2)$ and your engine freezes. A Double-Ended Queue (deque) allows $O(1)$ popping from the front via a doubly-linked list structure. This is non-negotiable for BFS.

Block 2: Asynchronous Execution in Real-Time Engines

When you execute `m03_gui.py`, you will watch the algorithms expand block by block. To animate a while loop that usually completes in microseconds, we cannot simply execute the function. Doing so would hijack the main thread and freeze Pygame's render pipeline.

We solve this using Python **Generators**.

```
def bfs_stepper(start: Point, goal: Point, grid_w: int, grid_h: int, walls: set[
    Point]):
    queue = deque([[start]])
    visited = {start}
    steps = 0

    while queue:
        path = queue.popleft()
        current = path[-1]
        steps += 1

        if current == goal:
            yield path, visited, True, steps # Yield completion state
            return

        # ... (neighbour expansion logic) ...

        yield path, visited, False, steps # Yield frame state
```

Engine Integration Breakdown: By replacing `return` with `yield`, the function yields its current state back to the Pygame main loop, pauses its own execution, allows Pygame to draw the UI and handle input events, and then resumes precisely where it left off on the next tick via the `next()` call. This is how you build asynchronous debuggers and spread heavy compute across multiple engine frames without invoking dangerous multithreading overhead.

The Synchronous Freeze (Thread Blocking): In the accompanying GUI script, you have the ability to toggle Synchronous Execution. When activated, the engine bypasses the generator and runs the search in a standard, blocking `while` loop.

- **The Result:** The algorithm executes sequentially from start to finish without yielding execution time back to the Pygame event pump.
- **The Consequence:** The UI completely freezes. The OS will likely flag the application as "Not Responding". No inputs are processed, and no frames are rendered until the search terminates. This strictly demonstrates why heavy AI computation must be time-sliced, delegated to worker threads, or managed via asynchronous coroutines.

Block 3: The Memory Reality Check (Data View)

Launch the GUI script and press **[V]** to toggle the Data Views.

The application runs on a 40x30 grid. That is 1,200 nodes.

- **Adjacency List:** Stores 1,200 keys, each holding a maximum of 4 orthogonal neighbours. Total elements: $\approx 6,000$. Highly manageable.
- **Adjacency Matrix:** Requires a 2D boolean array of 1200×1200 . That is **1.44 million entries**.

The GUI script explicitly refuses to render the full Adjacency Matrix, limiting the view to a 10x10 slice. Attempting to render 1.44 million boolean states to a screen every 16ms (60 FPS) will crash the application. When designing Gamee AI systems, you must respect $O(V^2)$ space complexity. Grids must use Adjacency Lists or dynamically compute neighbours on the fly.

Execution

Run the Pygame GUI. Draw a wall bisecting the screen.

1. Press **[B]** for BFS. Watch the frontier expand in a perfect, memory-heavy radial pattern. Observe the massive final step count in the sidebar.
2. Press **[C]** to clear, then **[D]** for DFS. Watch it plunge blindly along the wall. Observe how few steps it took, but notice that the path found is wildly sub-optimal.
3. Press **[F]** to toggle Synchronous/Blocking mode. Run BFS on a complex grid. Attempt to drag the application window or click. You will experience a total engine lockup until the algorithm finishes. This demonstrates the catastrophic failure of executing unbounded loops on the main thread.

Uninformed searches are the mathematical foundation of graph theory, but their lack of heuristics makes them unsuitable for real-time 3D pathfinding. In the next module, we implement A^* to solve this.

Glossary

- **Graph:** A mathematical structure used to model pairwise relations between objects. A graph is made up of **Nodes** (vertices, representing locations or states) and **Edges** (links or paths connecting the nodes).
- **Uninformed Search:** A “blind” search algorithm (like BFS or DFS) that has no additional information about states beyond the problem definition. It does not know if a node is closer or further from the goal; it only knows if it *is* the goal.
- **Breadth-First Search (BFS):** A search algorithm that explores the graph layer by layer, checking all immediate neighbours of a node before moving to the next depth level. It uses a **Queue (FIFO)**. Guarantees the shortest path in unweighted graphs but has a massive memory footprint.
- **Depth-First Search (DFS):** A search algorithm that plunges as deep as possible along a single branch before backtracking. It uses a **Stack (LIFO)**. Highly memory efficient, but does *not* guarantee the shortest path and can get trapped in deep trees.
- **Deque (Double-Ended Queue):** A data structure that allows elements to be added or removed from either the front or the back in $O(1)$ time. Absolutely mandatory for implementing BFS efficiently in Python.
- **$O(1)$ vs $O(N)$ Time Complexity:** $O(1)$ means an operation takes constant time, regardless of data size (e.g., looking up a dictionary key or popping from a deque). $O(N)$ means the time scales linearly with the data size (e.g., popping the first element of a standard list, which forces all N remaining elements to shift in memory).
- **Generator / Yield:** A Python feature that allows a function to return an intermediate value (yield) and pause its execution state, rather than destroying its local variables like a return statement. Crucial for slicing heavy algorithms across multiple frames in a game loop to prevent freezing.